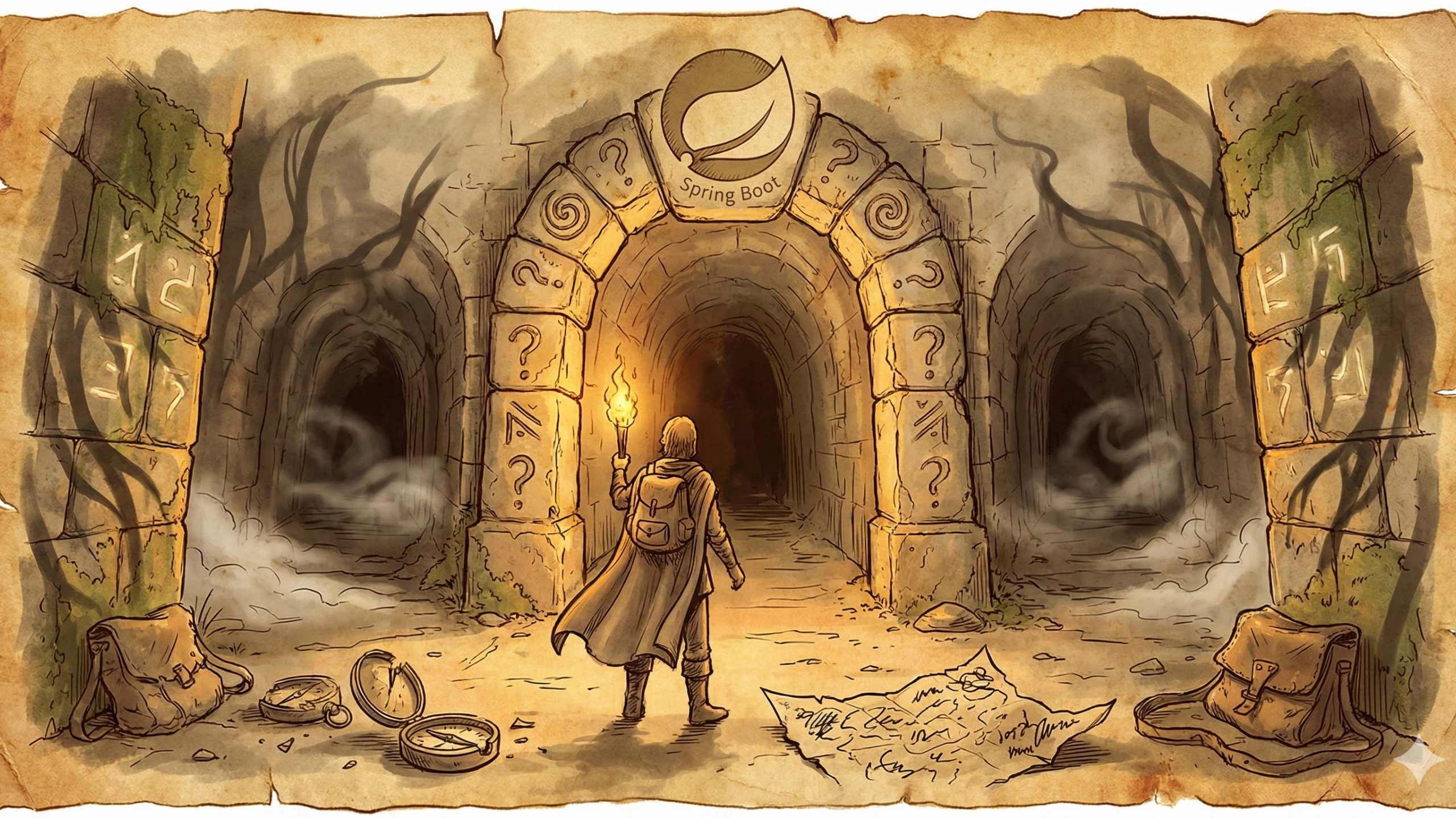

Spring Boot

Testing Spring Boot Applications Demystified

A Hero's Journey Through the Spring Boot Testing Labyrinth

Online Webinar 17th of December 2025

Philip Riecks - PragmaTech GmbH - @rieckpil

Participate During the Talk

Go to menti.com and use the code **7747 1181** to **anonymously** submit answers for the quizzes and add your questions during the talk.

Start with the first two questions:

- Despite having LLMs and Code Agents, do you still write your tests by hand?
- Do You Enjoy Writing Automated Tests?

At the end of the Menti, you can add your questions for the Q&A session.

Act 1: The Grand Entrance

Testing Spring Boot applications can feel like entering a labyrinth blindfolded:

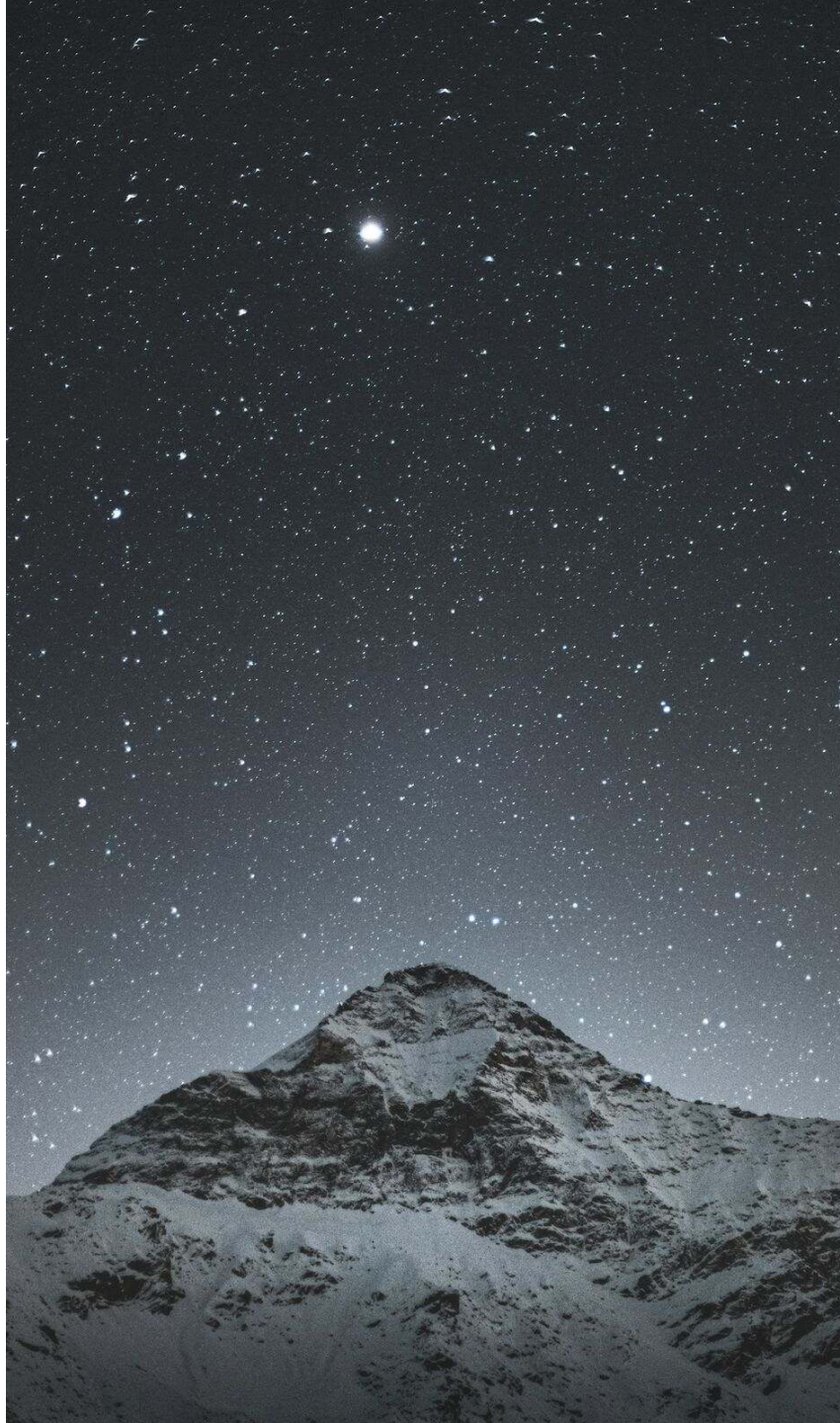
- Copying test configuration from AI/StackOverflow hoping it works
- The paradox of choice: `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `@MockBean`, etc.
- Fear of refactoring because tests break for the wrong reasons
- Tests written to satisfy coverage metrics, not increase productivity, confidence or catch bugs

My Overall Northstar for Automated Testing

Imagine seeing this pull request on a Friday afternoon:

How confident are you to merge this major Spring Boot upgrade and deploy it to production once the pipeline turns green?

Good tests don't just catch bugs - they give you **fast feedback** and **confident deployments**.



The Hero's Journey aka. Our Agenda

Goals For This Talk

- **Provide a clear mental map** for choosing between unit, slice, and integration tests so developers stop guessing which tool to use
- **Equip attendees with practical techniques** to speed up test suites and validate test quality
- **Build confidence to refactor fearlessly** by creating tests that catch real bugs, not just achieve coverage metrics

About Philip

- Self-employed developer from Herzogenaurach, Germany (Bavaria) 🍻
- Blogging & content creation with a focus on testing Java and specifically Spring Boot applications 🍃
- Founder of PragmaTech GmbH - Enabling Developers to Frequently Deliver Software with More Confidence 🚤
- Enjoys writing tests 🧪

Quest 1

The Unit Testing Guardian

The Swift Gatekeeper - Blocks Those Who Overcomplicate

Our Foundation: Spring Boot Starter Test

- The "Testing Swiss Army Knife"

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

- Batteries-included for testing by transitively including popular testing libraries
- Out-of-the-box dependency management to ensure compatibility

```
./mvnw dependency:tree
[INFO] ...
[INFO] +- org.springframework.boot:spring-boot-starter-test:jar:3.5.6:test
[INFO] |  +- org.springframework.boot:spring-boot-test:jar:3.5.6:test
[INFO] |  +- org.springframework.boot:spring-boot-test-autoconfigure:jar:3.5.6:test
[INFO] |  +- com.jayway.jsonpath:json-path:jar:2.9.0:test
[INFO] |  +- jakarta.xml.bind:jakarta.xml.bind-api:jar:4.0.2:test
[INFO] |  |  \- jakarta.activation:jakarta.activation-api:jar:2.1.4:test
[INFO] |  +- net.minidev:json-smart:jar:2.5.2:test
[INFO] |  |  \- net.minidev:accessors-smart:jar:2.5.2:test
[INFO] |  |     \- org.ow2.asm:asm:jar:9.7.1:test
[INFO] |  +- org.assertj:assertj-core:jar:3.27.4:test
[INFO] |  |  \- net.bytebuddy:byte-buddy:jar:1.17.7:test
[INFO] |  +- org.awaitility:awaitility:jar:4.3.0:test
[INFO] |  +- org.hamcrest:hamcrest:jar:3.0:test
[INFO] |  +- org.junit.jupiter:junit-jupiter:jar:5.12.2:test
[INFO] |  |  +- org.junit.jupiter:junit-jupiter-api:jar:5.12.2:test
[INFO] |  |  |  +- org.junit.platform:junit-platform-commons:jar:1.12.2:test
[INFO] |  |  |  \- org.apiguardian:apiguardian-api:jar:1.1.2:test
[INFO] |  |  +- org.junit.jupiter:junit-jupiter-params:jar:5.12.2:test
[INFO] |  |  \- org.junit.jupiter:junit-jupiter-engine:jar:5.12.2:test
[INFO] |  |     \- org.junit.platform:junit-platform-engine:jar:1.12.2:test
[INFO] |  +- org.mockito:mockito-core:jar:5.16.0:test
[INFO] |  |  +- net.bytebuddy:byte-buddy-agent:jar:1.17.7:test
[INFO] |  |  \- org.objenesis:objenesis:jar:3.3:test
[INFO] |  +- org.mockito:mockito-junit-jupiter:jar:5.16.0:test
[INFO] |  +- org.skyscreamer:jsonassert:jar:1.5.3:test
[INFO] |  |  \- com.vaadin.external.google:android-json:jar:0.0.20131108.vaadin1:test
[INFO] |  +- org.springframework:spring-core:jar:6.2.11:compile
[INFO] |  |  \- org.springframework:spring-jcl:jar:6.2.11:compile
[INFO] |  +- org.springframework:spring-test:jar:6.2.11:test
[INFO] |  \- org.xmlunit:xmlunit-core:jar:2.10.4:test
```

What's Inside the Testing Swiss Army Knife?

- **JUnit** (currently 5, later 6): Java's de-facto standard testing framework and foundation.
- **Mockito**: Creating mock objects to simulate dependencies and verify interactions.
- **AssertJ**: Provides fluent, chainable, and readable assertions.
- **Hamcrest**: Offers flexible matchers for creating custom assertions.
- **JSONAssert**: Compares JSON strings with flexible matching options.
- **JsonPath**: Extracts and queries data from JSON similar to XPath.
- **XMLUnit**: Compares and validates XML documents.
- **Awaitility**: Handles asynchronous testing with fluent conditions.

Unit Testing Spring Boot Applications 101

- **Core Concept**: Test individual components (classes, methods) in complete isolation from their dependencies.
- **Confidence Gained**: Provides logarithmic verifications, ensuring that the smallest parts of your code work as expected under various conditions.
- **Best Practices**: Focus on a single unit of work.
- **Pitfalls**: Requires a well-thought-out class design. Poor design can lead to testing overly complex "god classes," making tests difficult to write and maintain.
- **Tools**: JUnit (or Spock, TestNG, etc.), Mockito and assertion libraries like AssertJ or Hamcrest.

Unit Testing has Limits

Consider this sample REST controller, what could we verify with a unit test?

```java
@RestController
@RequestMapping("/api/customers")
public class CustomerController {

  private final CustomerService customerService;

  public CustomerController(CustomerService customerService) {
    this.customerService = customerService;
  }

  @PostMapping
  public ResponseEntity<Void> createNewCustomer(@Validated CustomerCreationRequest payload, UriComponentsBuilder uriBuilder) {

    String customerId = customerService.createNewCustomer(payload.firstName());

    UriComponents uriComponents = uriBuilder
      .path("/api/customers/{id}")
      .buildAndExpand(customerId);

    return ResponseEntity.created(uriComponents.toUri()).build();
  }
}
```

```java
@ExtendWith(MockitoExtension.class)
class CustomerControllerUnitTests {

  @Mock
  private CustomerService customerService;

  @InjectMocks
  private CustomerController customerController;

  @Test
  void shouldCreateCustomerWhenPayloadRequestIsValid() {
    when(customerService.createNewCustomer(anyString()))
      .thenReturn("42");

    ResponseEntity<Void> result = customerController.createNewCustomer(
      new CustomerCreationRequest("Java", "Duke", "duke@spring.io"),
      UriComponentsBuilder.newInstance()
    );

    assertThat(result.getStatusCode().value())
      .isEqualTo(201);
    assertThat(result.getHeaders().getLocation().toString())
      .isEqualTo("/api/customers/42");
  }
}
```

Things We Can't Cover with a Unit Test

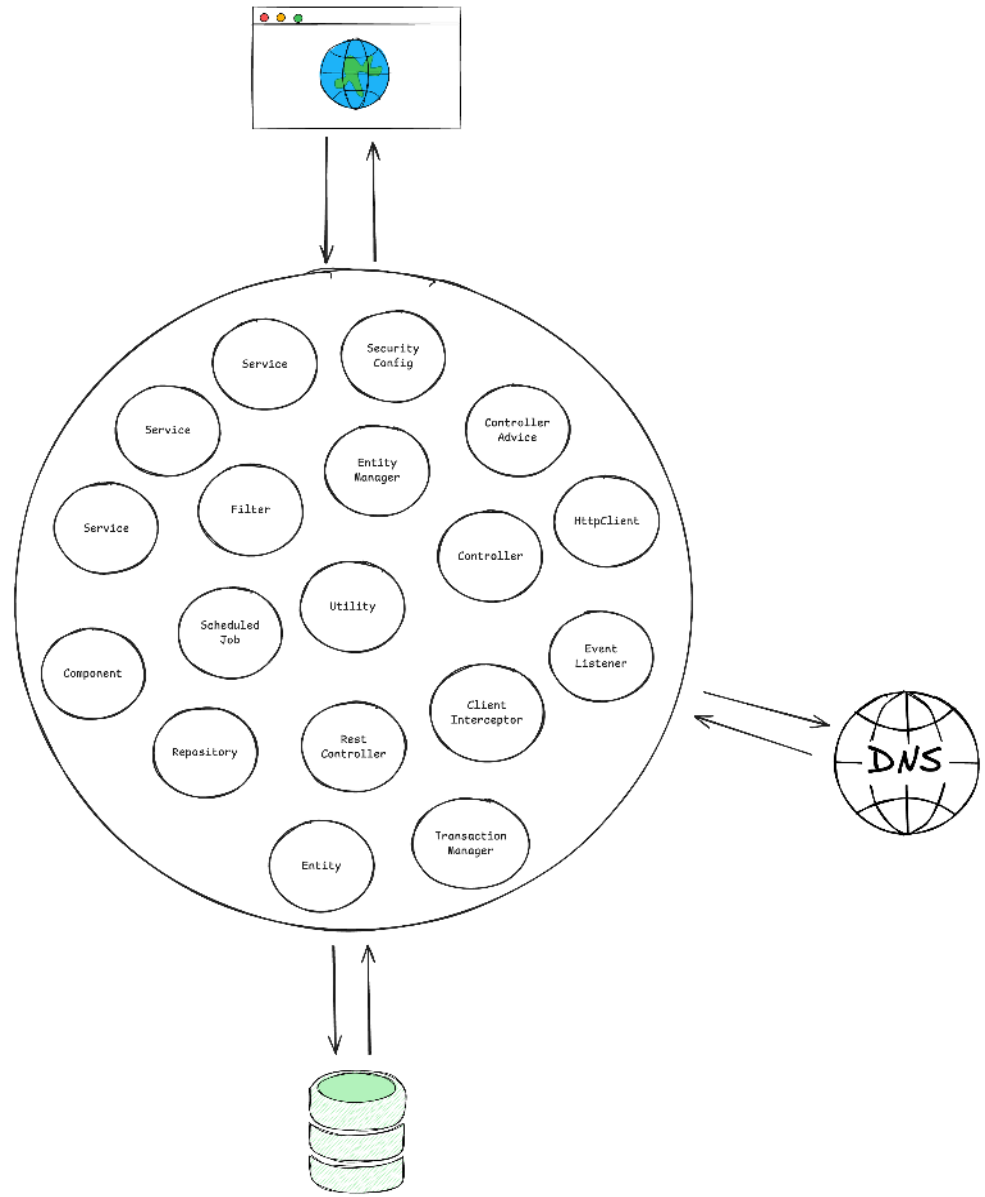
- **Request Mapping**: Does HTTP GET `/api/customers/{id}` actually resolve to our desired method?
- **Validation**: Will incomplete request bodys result in a 400 bad request or return an accidental 201?
- **Serialization**: Are we JSON objects serialized and deserialized correctly?
- **Headers**: Are we setting `Content-Type` or custom headers correctly?
- **Security**: Are we Spring Security configuration and other authorization checks enforced?

Quest 2

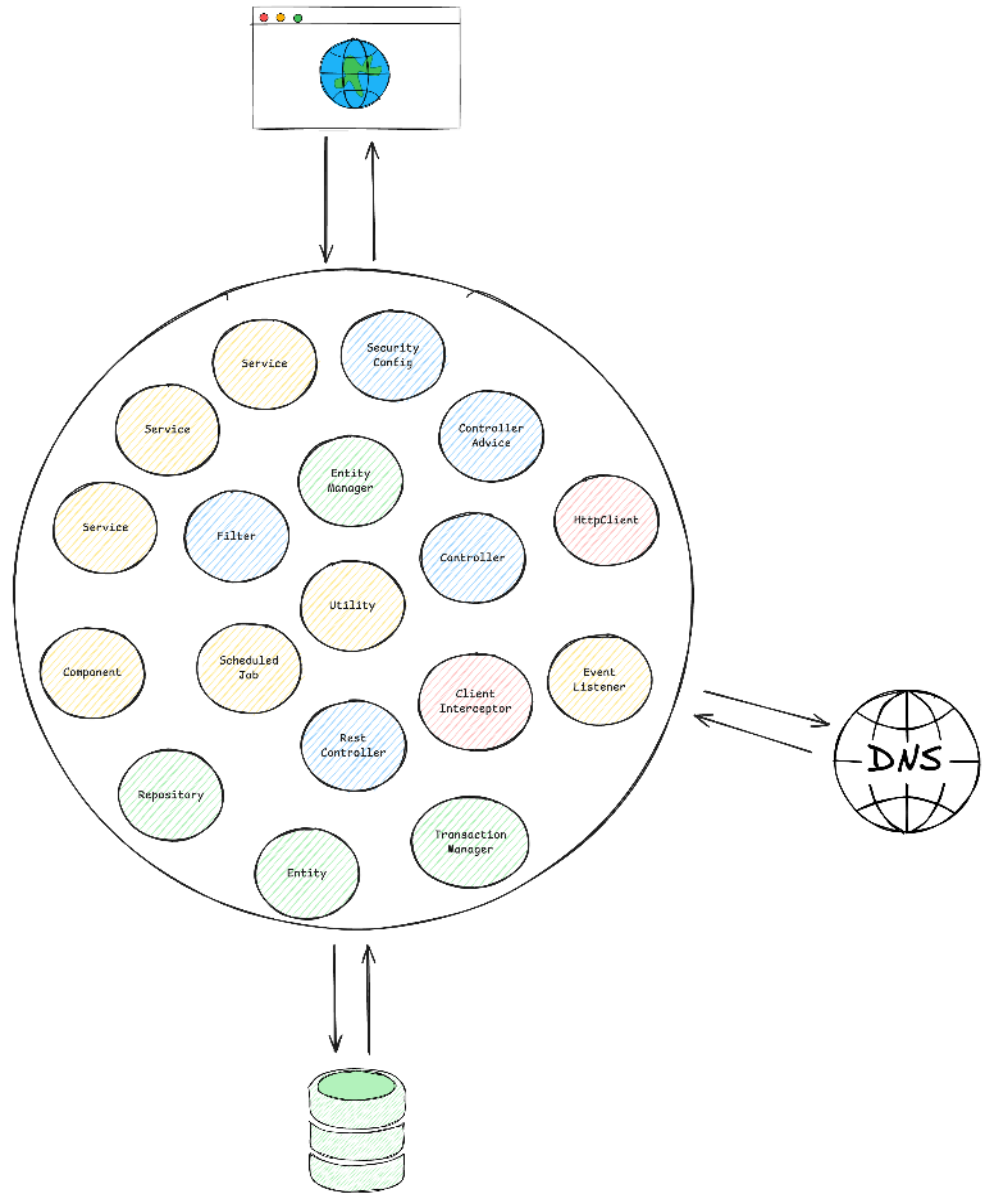
The Slice Testing Hydra

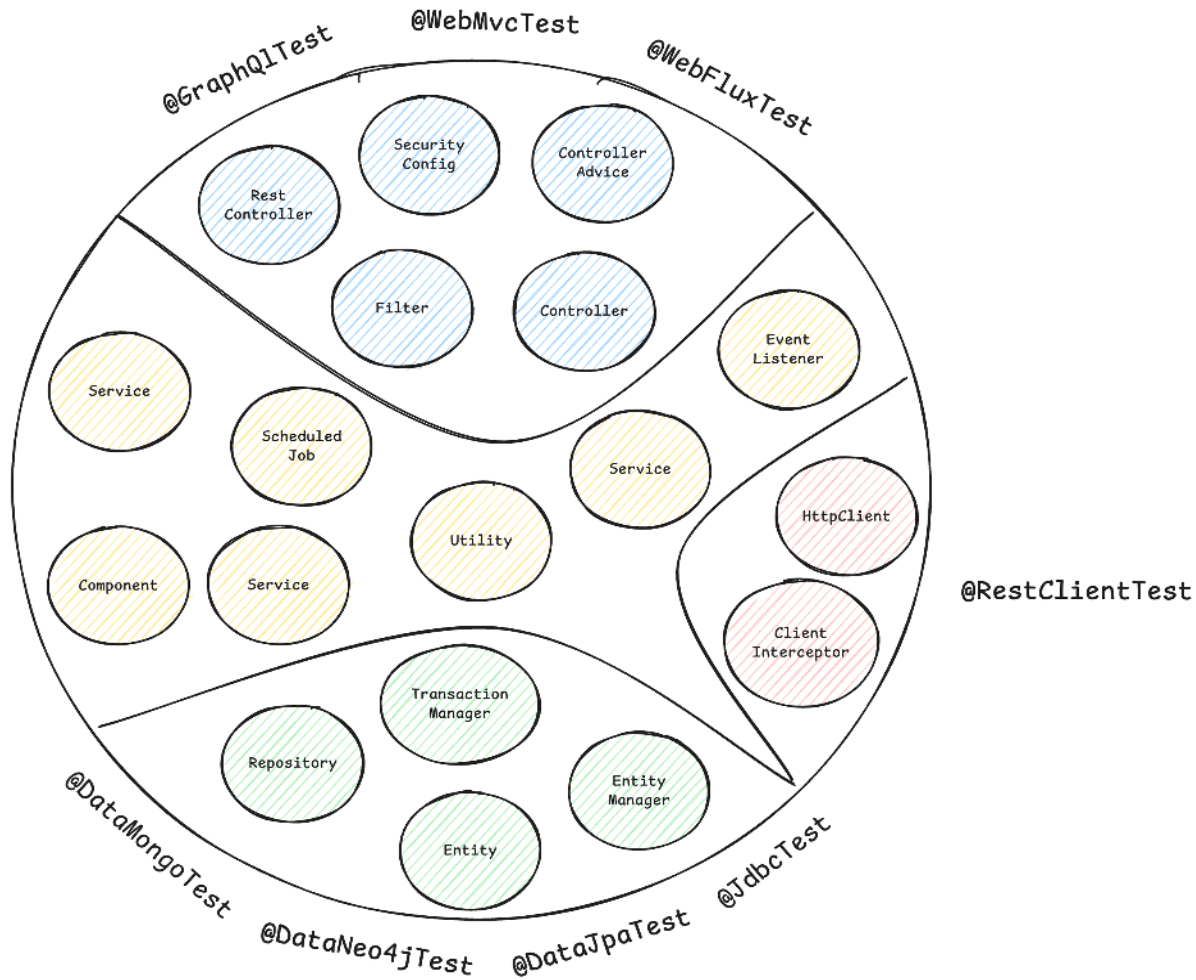
Multiple Heads, Each Guarding a Layer

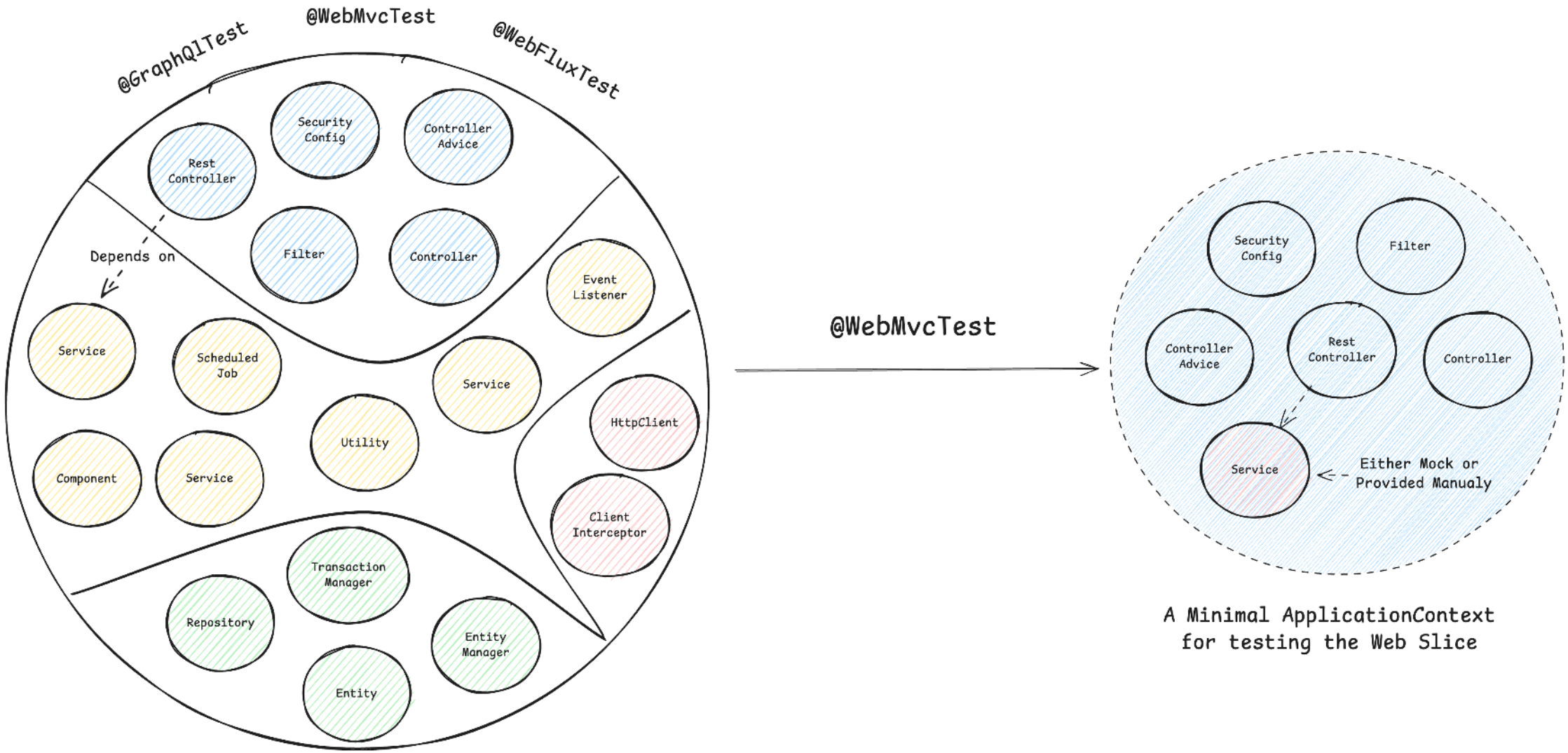
A Typical
Spring ApplicationContext
with different
Spring Beans



A Typical
Spring ApplicationContext




@GraphQlTest
@WebMvcTest
@WebFluxTest
Security Config
Controller Advice
Rest Controller
Filter
Controller
Event Listener
Service
Scheduled Job
Service
HttpClient
Utility
Component
Service
Client Interceptor
@RestClientTest
Transaction Manager
Repository
Entity Manager
Entity
@DataMongoTest
@DataNeo4jTest
@DataJpaTest
@JdbcTest


@GraphQlTest
@WebMvcTest
@WebFluxTest
Security Config
Controller Advice
Rest Controller
Filter
Controller
Depends on
Event Listener
Service
Scheduled Job
Service
HttpClient
Utility
Component
Service
Client Interceptor
Transaction Manager
Repository
Entity Manager
Entity
@WebMvcTest
Security Config
Filter
Controller Advice
Rest Controller
Controller
Service
Either Mock or Provided Manualy
A Minimal ApplicationContext for testing the Web Slice

Spring Boot Test Slice Example: `@WebMvcTest`

```
1  @WebMvcTest(CustomerController.class)
2  @Import(SecurityConfig.class)
3  class CustomerControllerTest {
4
5    @Autowired
6    private MockMvc mockMvc;
7
8    @MockitoBean
9    private CustomerService customerService;
10
11   @Test
12   @WithMockUser
13   void shouldReturnLocationOfNewlyCreatedCustomer() throws Exception {
14     // ...
15   }
16 }
```

Common Test Slices

- `@WebMvcTest` / `@WebFluxTest` - Controller layer
- `@DataJpaTest` / `@JdbcTest` - Persistence layer
- `@JsonTest` - JSON serialization/deserialization
- `@RestClientTest` - RestTemplate testing
- etc.

@DataCouchbaseTest

@DataLdapTest

@RestClientTest

@JsonTest

@JooqTest

@WebFluxTest

@DataRedisTest

@WebMvcTest

@DataMongoTest

@DataCassandraTest

@GraphQlTest

@DataNeo4jTest

@WriteYourOwn*

@DataJpaTest

@SqsTest

@JdbcTest

@DataElasticsearchTest

Quest 3

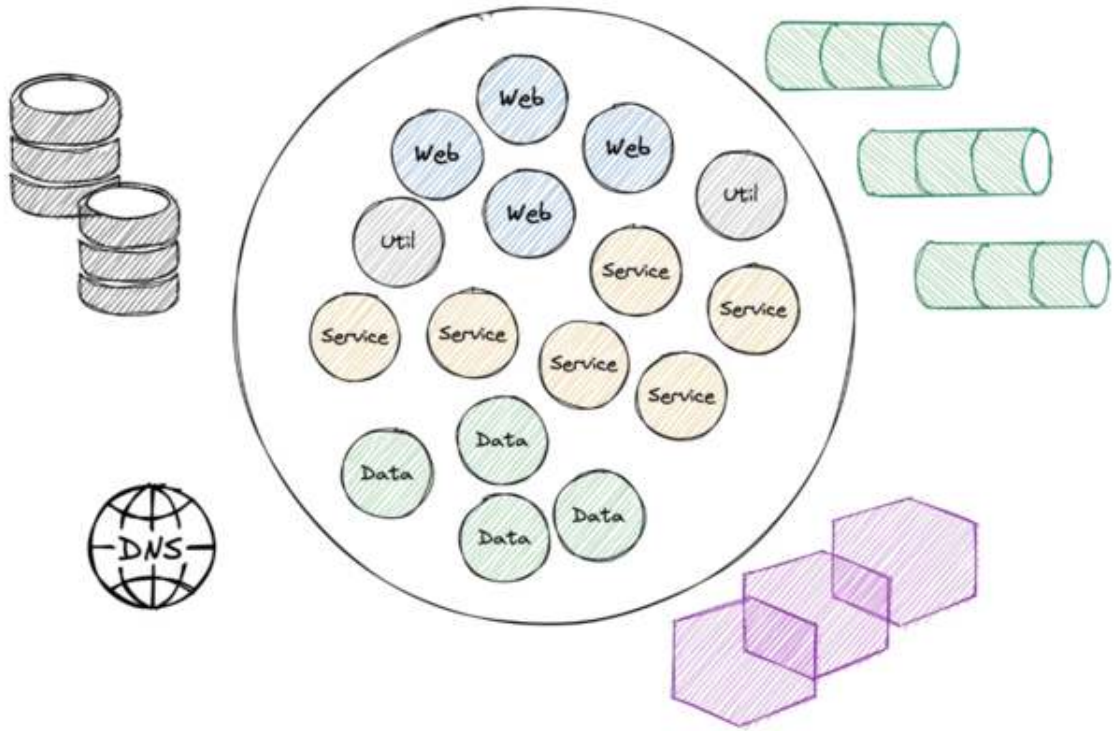
The Integration Testing Dragon

Guards the Full Treasure - but Demands Patience

```java
@SpringBootTest
class ApplicationTest {

  @Test
  void contextLoads() {
  }

}
```



Integration Testing Spring Boot Applications 101

- **Core Concept**: Start the entire Spring application context, often on a random local port, and test the application through its external interfaces (e.g., REST API).
- **Confidence Gained**: Validates the integration of all internal components working together as a complete application.
- **Best Practices**: Use `@SpringBootTest` to run the app on a local port.
- **Pitfalls**: Slower to run than unit or sliced tests. Managing the lifecycle of dependent services can be complex.
- **Tools**: JUnit, Mockito, Spring Test, Spring Boot, Testcontainers, WireMock (for mocking external HTTP services), Selenium (for browser-based UI testing)

Starting the Entire `ApplicationContext`

- **Problem #1**: How to ensure surrounding infrastructure (e.g. database, queues, etc.) is present?
- **Problem #2**: How to handle HTTP communication from our application to remote services?
- **Problem #3**: How to keep our build time at a reasonable duration?

Provide External Infrastructure with Testcontainers (Problem #1)

Running infrastructure components (databases, message brokers, etc.) in Docker containers for our tests becomes a breeze with Testcontainers:

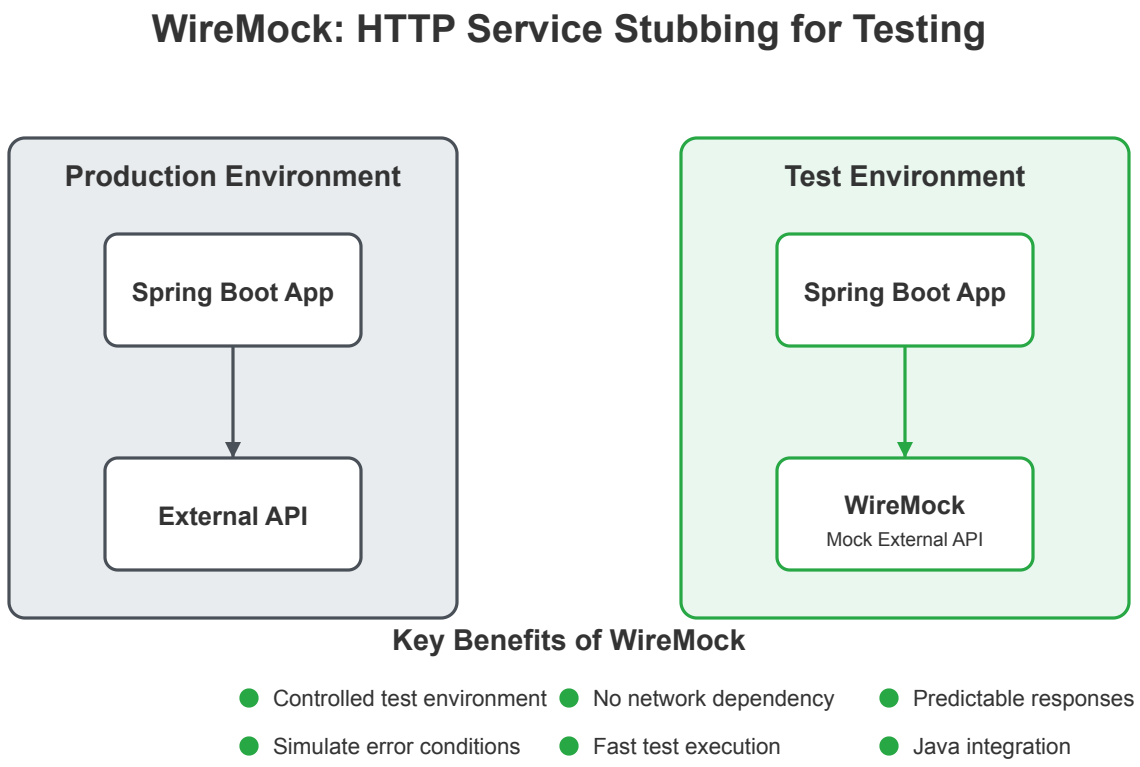
```
@Container
@ServiceConnection
static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine")
  .withDatabaseName("testdb")
  .withUsername("test")
  .withPassword("test")
  .withInitScript("init-postgres.sql");
```

This gives us an ephemeral PostgreSQL database for our tests:

```
$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS          PORTS                                                NAMES
a958ee2887c6   postgres:16-alpine             "docker-entrypoint.s…"   10 seconds ago   Up 9 seconds    0.0.0.0:32776->5432/tcp, [::]:32776->5432/tcp        affectionate_cannon
ad0f804068dc   testcontainers/ryuk:0.12.0     "/bin/ryuk"              10 seconds ago   Up 9 seconds    0.0.0.0:32775->8080/tcp, [::]:32775->8080/tcp        testcontainers-ryuk-1f9f76a6-46d4-4e19-85c1-e8364da12804
```

Stub External HTTP Services with WireMock (Problem #2)

Consider WireMock to stub external HTTP services during tests.



Using WireMock for Integration Tests

- Run as in-memory service or Docker container to simulate connected HTTP services
- Override HTTP clients to connect to the WireMock server during tests

```
TestPropertyValues.of(
  "clients.open-library.base-url=http://localhost:"+ wireMockServer.port())
  .applyTo(applicationContext);
```

```
wireMockServer.stubFor(
  get(urlPathEqualTo("/api/books/" + isbn))
    .willReturn(aResponse()
      .withHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)
      .withBodyFile("book-response-success.json"))
);
```

Starting the Entire Spring Context - Version 1

- We access the application over HTTP like a user, the test and context run in separate threads (no `@Transactional` rollback), requires HTTP authentication

```
1  @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
2  class ApplicationServletContainerIT {
3
4    @LocalServerPort
5    private int port; // <-- we're running on a real port
6
7    @Test
8    void contextLoads(@Autowired WebTestClient webTestClient) {
9      webTestClient
10       .get()
11       .uri("/api/customers")
12       .header("Authorization", "Basic " + Base64.getEncoder().encodeToString("user:dummy".getBytes()))
13       .exchange()
14       .expectStatus()
15       .isOk();
16   }
17 }
```

Starting the Entire Spring Context - Version 2

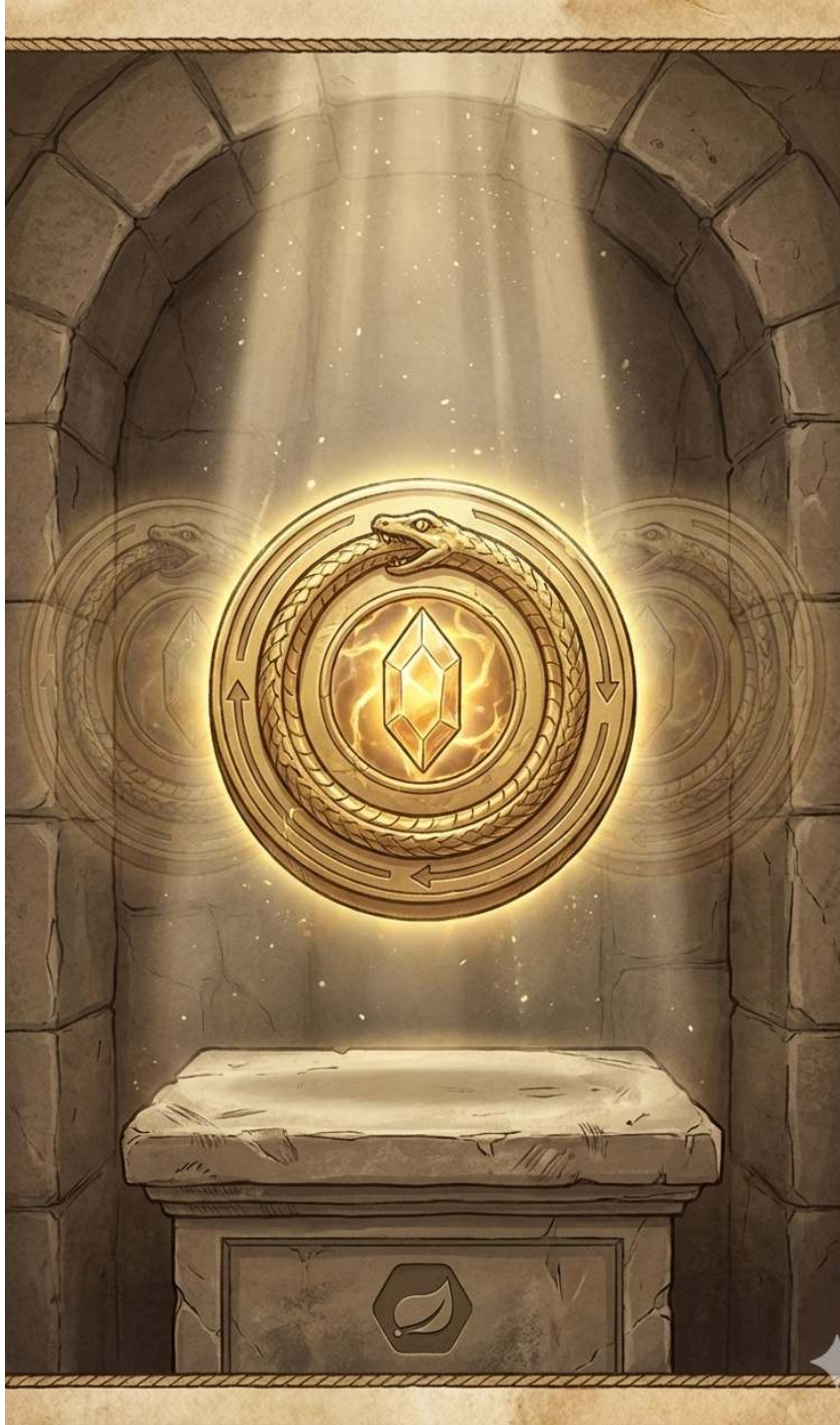
- The test and the context run in the same thread, hence we can rollback with `@Transactional` and simply override the security context with `@WithMockUser`

```
1 @SpringBootTest
2 // which is @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK)
3 @AutoConfigureMockMvc
4 class ApplicationMockWebIT {
5
6   // @LocalServerPort
7   // private int port; <-- this would fail the test, there is no local port occupied
8
9   @Test
10  @WithMockUser
11  void givenCustomersThenReturnListForAuthenticatedUser(@Autowired MockMvc mockMvc) throws Exception {
12    mockMvc
13      .perform(get("/api/customers")
14        .header(ACCEPT, APPLICATION_JSON))
15      .andExpect(status().is(200))
16      .andExpect(content().contentType(APPLICATION_JSON))
17      .andExpect(jsonPath("$.size()", is(1)));
18  }
19 }
```

Quest Item 1

The Caching Amulet

Helps You Reuse What You Already Built



Integration Testing - The Need for Speed

- **The Problem:** Integration tests require a started & initialized Spring `ApplicationContext`, which slows down the build
- **The Solution:** Spring Test `TestContext` Caching – stores an already started Spring `ApplicationContext` for reuse
- This feature is part of Spring Test (included in every Spring Boot project via `spring-boot-starter-test`)

Example of speed improvement:

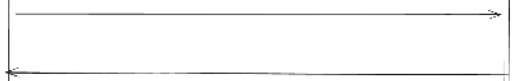
Test run time: 3506ms

OrderIT
requiring context
configuration with the
hash code #327321289

PaymentIT
requiring context
configuration with the
hash code #327321289

CheckoutIT
requiring context
configuration with the
hash code #565212314

Context cache lookup



Spring TestContext Context Cache

Cache Key

Cache Value

no cached context,
tests needs
to start the context

Test run time: 3506ms

OrderIT
requiring context
configuration with the
hash code #327321289

Test run time: 406ms

PaymentIT
requiring context
configuration with the
hash code #327321289

test gets cached context and
finishes the test very fast

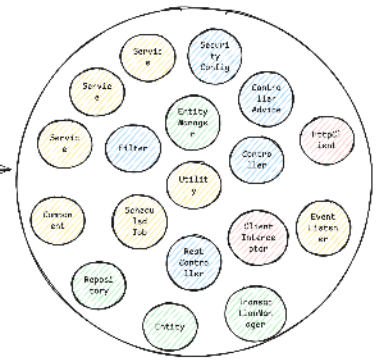
CheckoutIT
requiring context
configuration with the
hash code #565212314

Spring TestContext Context Cache

Cache Key

Cache Value

#327321289



Test run time: 3506ms

OrderIT
requiring context
configuration with the
hash code #327321289

Test run time: 406ms

PaymentIT
requiring context
configuration with the
hash code #327321289

Test run time: 4506ms

CheckoutIT
requiring context
configuration with the
hash code #565212314

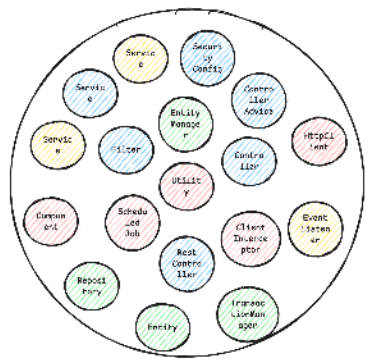
Spring TestContext Context Cache

Cache Key

Cache Value

#327321289

#565212314



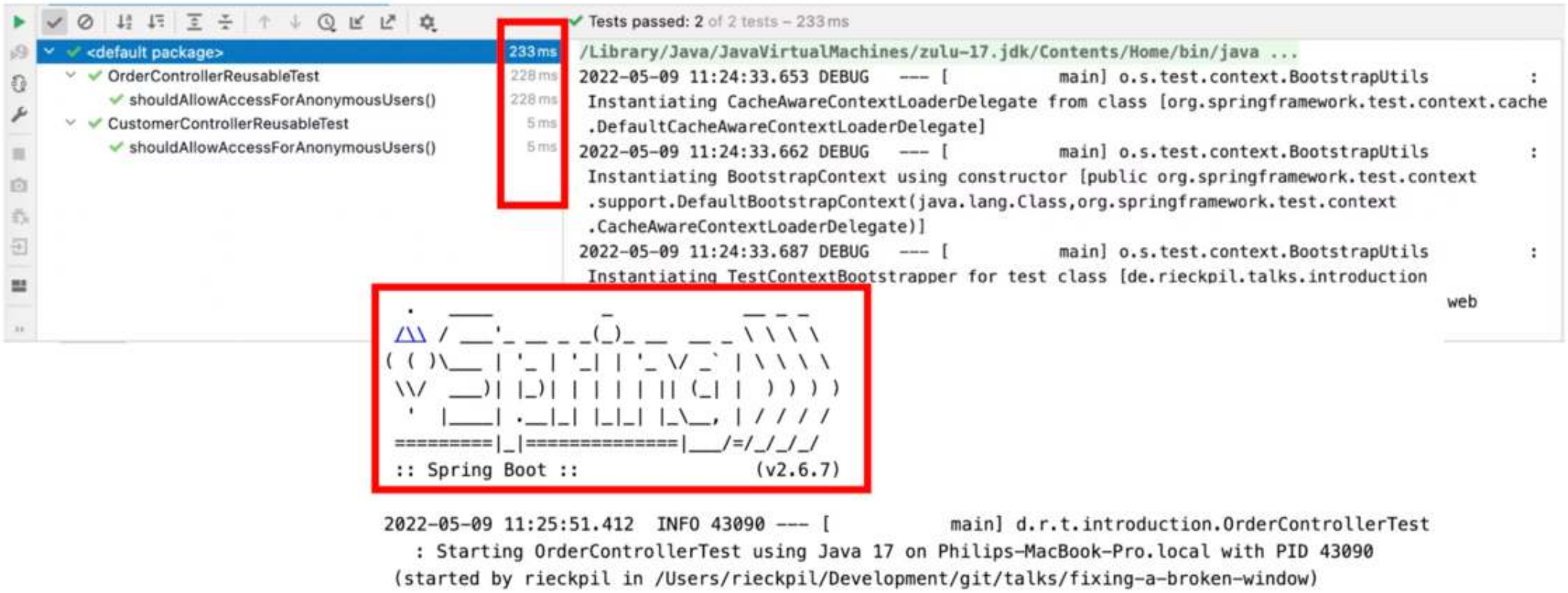
How the Cache Key is Built

```
1 // DefaultContextCache.java
2 private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3   Collections.synchronizedMap(new LruCache(32, 0.75f));
```

The following information is part of the Cache Key (`MergedContextConfiguration`):

- activeProfiles (`@ActiveProfiles`)
- contextInitializersClasses (`@ContextConfiguration`)
- propertySourceLocations (`@TestPropertySource`)
- propertySourceProperties (`@TestPropertySource`)
- contextCustomizer (`@MockitoBean` , `@MockBean` , `@DynamicPropertySource` , ...)
- etc.

Detect Context Restarts - Visually



Detect Context Restarts - with Logs

```
<logger name="org.springframework.test.context" level="TRACE" />
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [           main] org.springframework.test.context.cache   : Spring test
 ApplicationContext cache statistics: [DefaultContextCache@68e62b3b size = 1, maxSize = 32, parentContextCount = 0,
 hitCount = 11, missCount = 1]
2022-05-05 14:27:38.136 DEBUG 42500 --- [           main] tractDirtiesContextTestExecutionListener : After test class:
 context [DefaultTestContext@325bb9a6 testClass = ApplicationIT, testInstance = [null], testMethod = [null], testException
  = [null], mergedContextConfiguration = [WebMergedContextConfiguration@1d12b024 testClass = ApplicationIT, locations =
 '{}', classes = '{class de.rieckpil.talks.Application}', contextInitializerClasses = '[]', activeProfiles = '{}',
 propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context
 .SpringBootTestContextBootstrapper=true, server.port=0}', contextCustomizers = set[org.springframework.boot.test.context
 .filter.ExcludeFilterContextCustomizer@5215cd9a, org.springframework.boot.test.json
 .DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@9257031, org.springframework.boot.test
```

Detect Context Restarts - with Tooling

An open-source Spring Test utility that provides visualization and insights for Spring Test execution, with a focus on Spring context caching statistics.

Overall goal: Identify optimization opportunities in your Spring Test suite to speed up your builds and ship to production faster and with more confidence.

The Final Boss

Developers tend to consult AI/StackOverflow for integration test issues and often copy advice from the internet without knowing the implications:

```
1 @SpringBootTest
2 @DirtiesContext
3 // this instructs Spring to remove the context from the cache
4 // and rebuild a new context on every request
5 public abstract class AbstractIntegrationTest {
6
7 }
```

The setup above will **disable** the context caching feature and slow down the builds significantly!

New in Spring Framework 7: Pausing Contexts

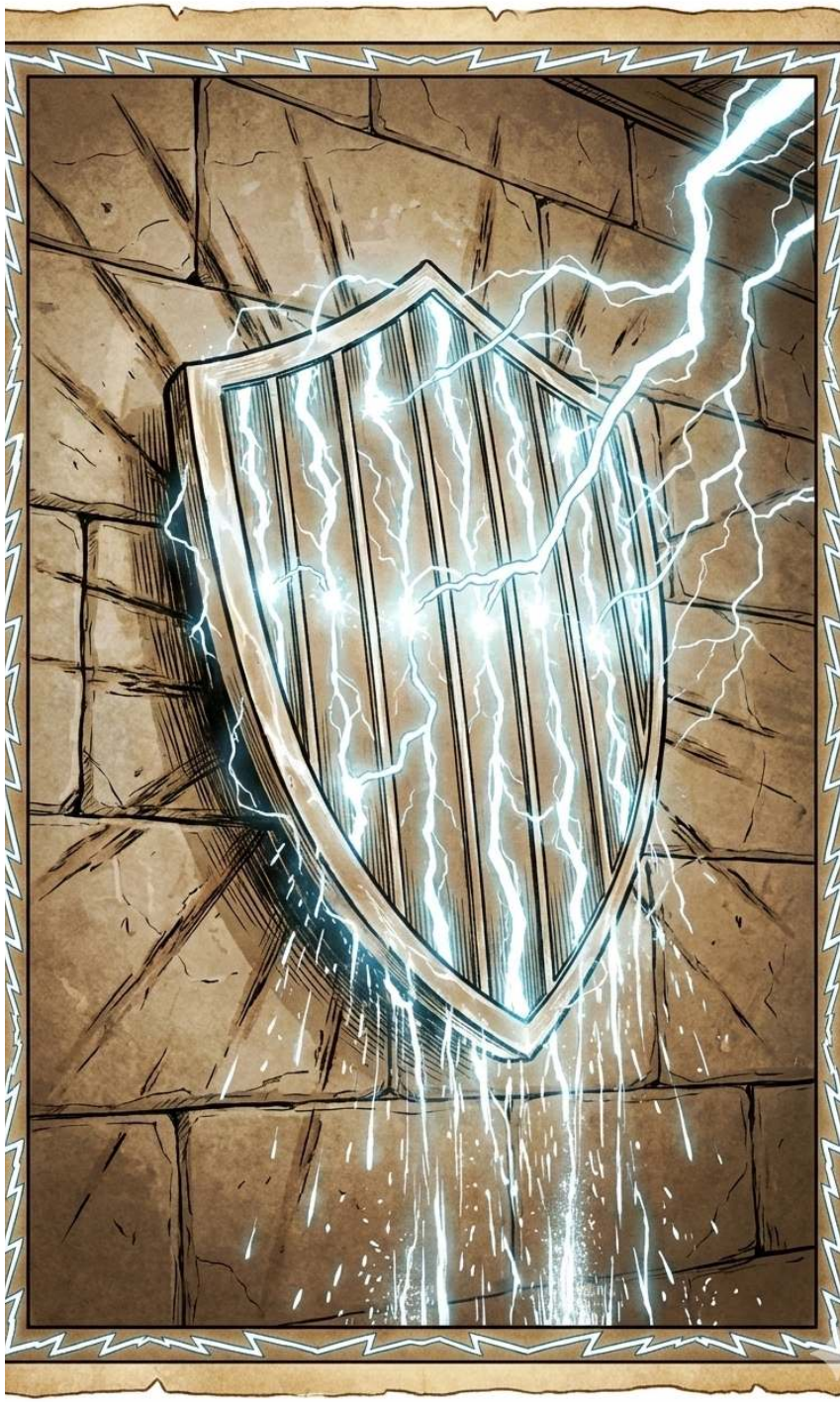
See Release Notes von Spring Framework 7.0.0 M7.

> Pausing of Test Application Contexts
>
> The Spring TestContext framework is caching application context instances within test suites for faster runs. As of Spring Framework 7.0, we now pause test application contexts when
> they're not used.
>
> This means an application context stored in the context cache will be stopped when it is no longer actively in use and automatically restarted the next time the context is retrieved from the cache.
>
> Specifically, the latter will restart all auto-startup beans in the application context, effectively restoring the lifecycle state.

Quest Item 2

The Lightning Shield

Many Cores, One Goal



Test Parallelization

Goal: Reduce build time and get faster feedback

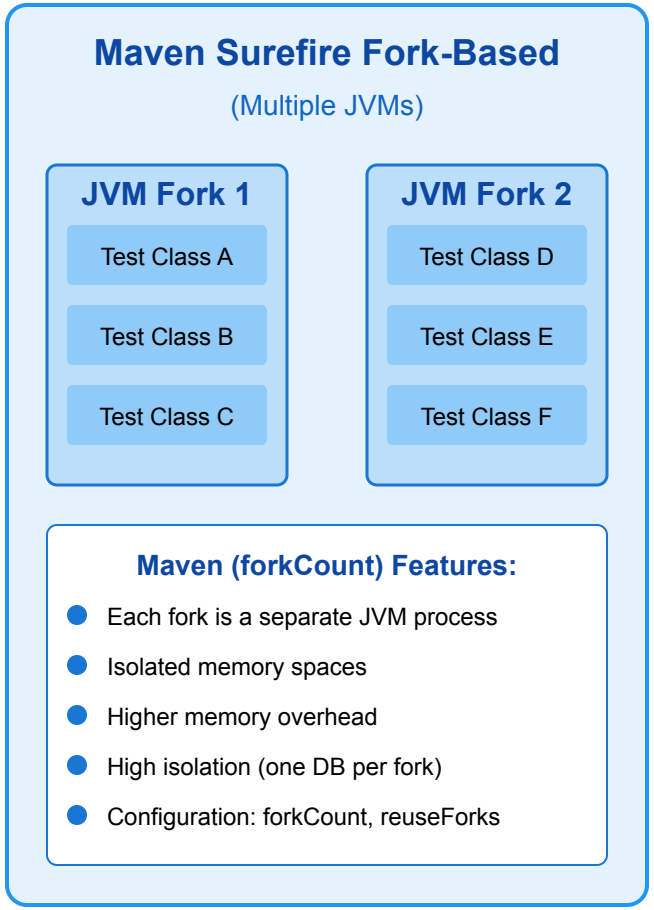
Requirements:

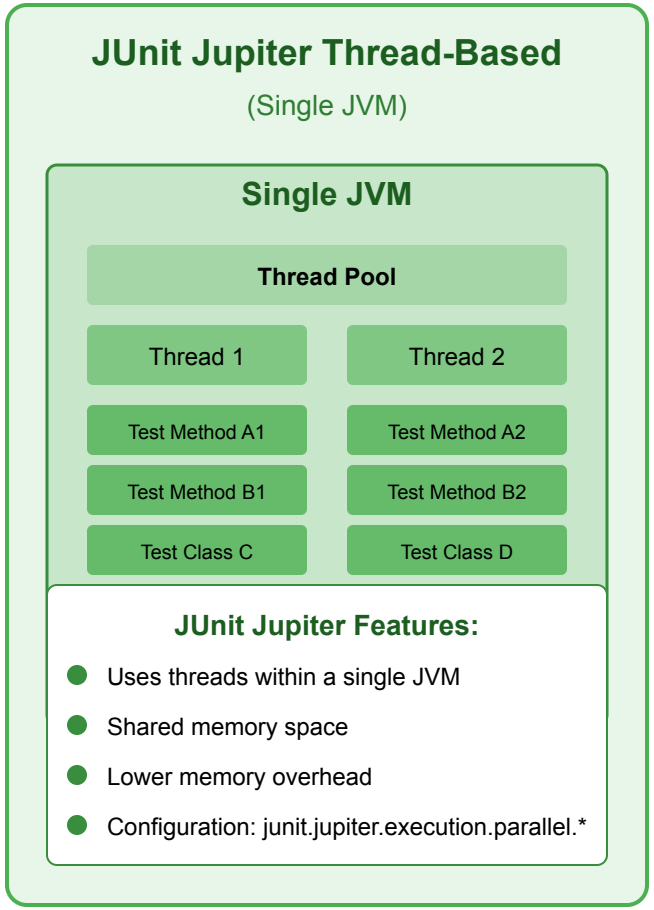
- No shared state
- No dependency between tests and their execution order
- No mutation of global state

Two ways to achieve this:

- Fork a new JVM with Surefire/Failsafe (or for the Gradle test task) and let it run in parallel
- Use JUnit Jupiter's parallelization mode and let it run in the same JVM with multiple threads

Java Test Parallelization Options





Reuse Containers with Testcontainers

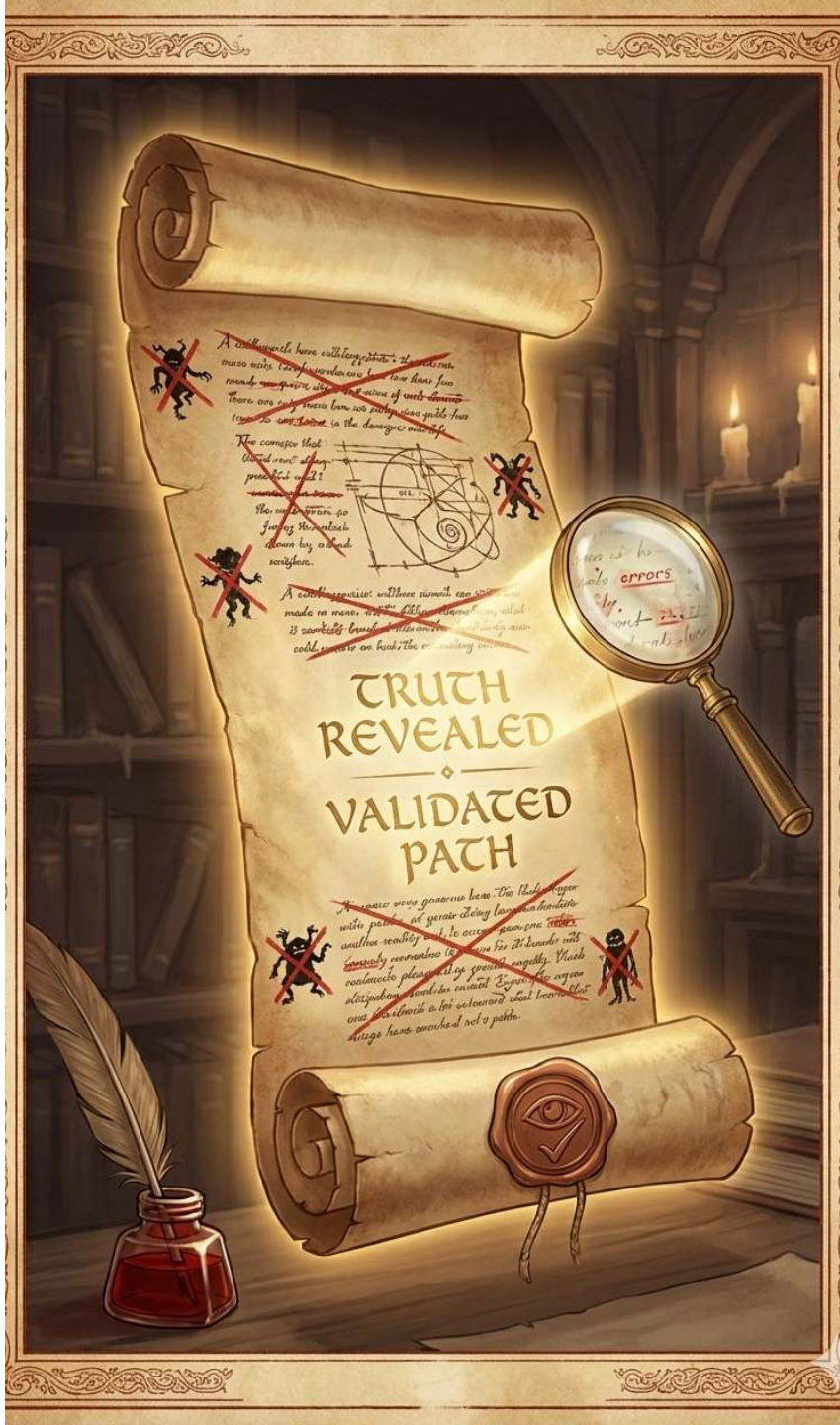
- Bootstrapping new containers can take a significant amount of time
- Enable container reuse in Testcontainers when possible: `.withReuse(true)`
- Singleton containers per test run are preferable to `@Testcontainers` (container per test class)
- Speed up container startup with e.g. predefined database snapshots

```
1 private static PostgreSQLContainer<?> postgresModule = new PostgreSQLContainer<>("myteampostgres:42")
2   .withDatabaseName("testdb")
3   .withUsername("testuser")
4   .withPassword("testpass");
5
6 static {
7   postgresModule.start();
8 }
```

Quest Item 3

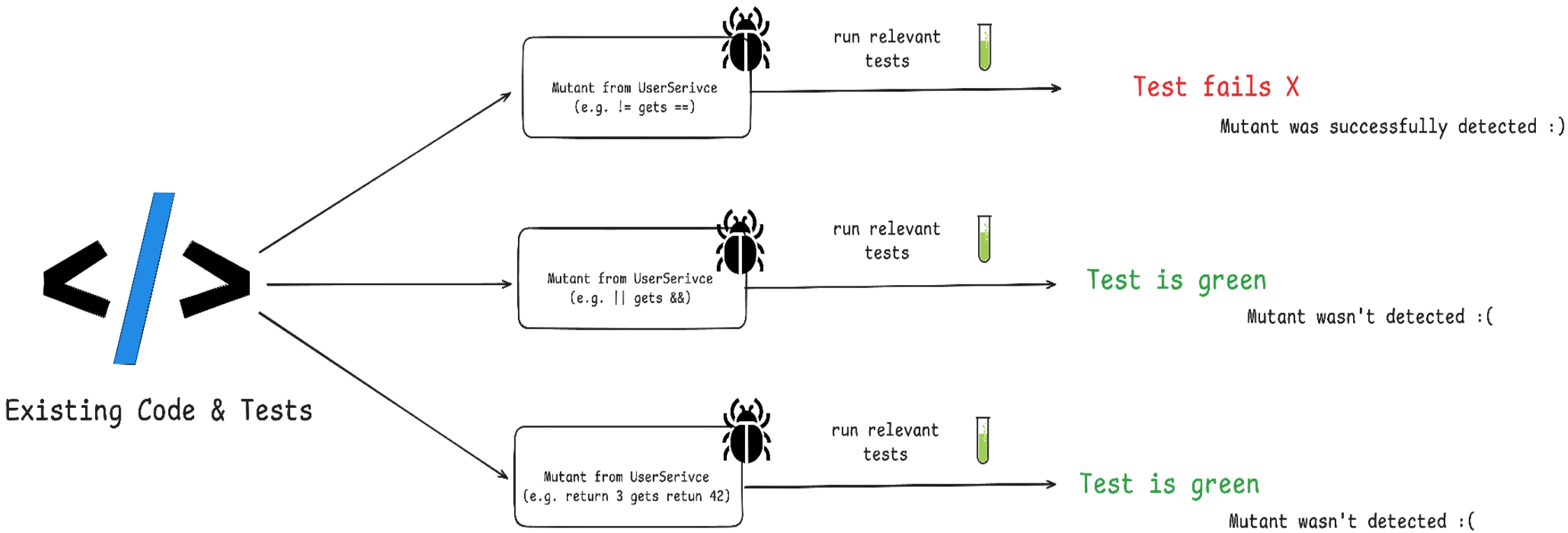
The Scroll of Truth

Coverage Lies, Mutants Don't



Introducing: Mutation Testing

- Having high code coverage might give you a **false sense of security**
- Mutation Testing with PIT
- Beyond Line Coverage: Traditional tools like JaCoCo show which code runs during tests, but PIT verifies if our tests actually detect when code behaves incorrectly by introducing "**mutations**" to our source code.
- Quality Guarantee: PIT automatically **modifies our code** (changing conditionals, return values, etc.) to ensure our tests fail when they should, **revealing blind spots** in seemingly comprehensive test suites.


Existing Code & Tests
Mutant from UserSerivce
(e.g. != gets ==)
run relevant
tests
Test fails X
Mutant was successfully detected :)
Mutant from UserSerivce
(e.g. || gets &&)
run relevant
tests
Test is green
Mutant wasn't detected :(
Mutant from UserSerivce
(e.g. return 3 gets retun 42)
run relevant
tests
Test is green
Mutant wasn't detected :(

Act 5: The Triumphant Exit

- Spring Boot applications come with batteries-included and excellent testing support
- We've completed three main quests:
  - Unit testing
  - Sliced testing
  - Integration testing
- Three core quest items help us to speed up and validate our tests:
  - Context Caching
  - Test Parallelization
  - Mutation Testing

What's Next?

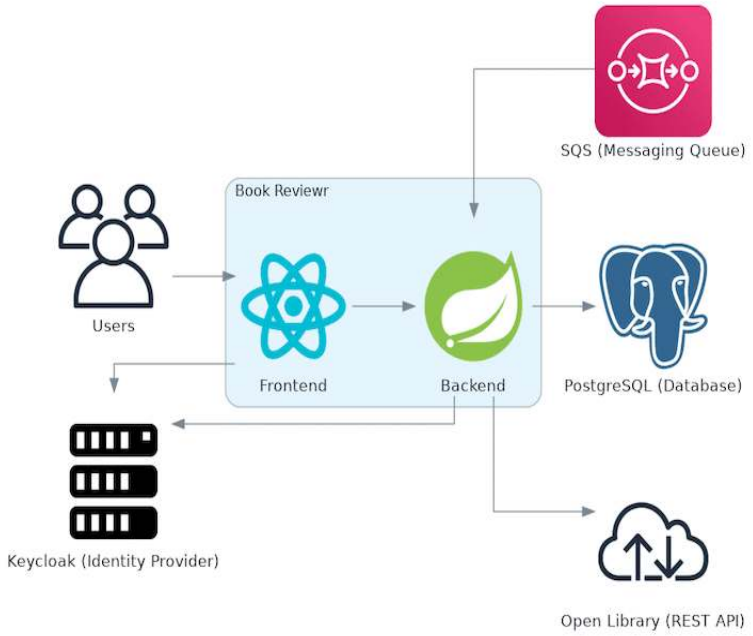
Testing is a team sport, make sure your whole team levels up together!

- Free meetup sessions for companies/teams:
  - Stop Fighting Your Spring Boot Tests
  - Testing Spring Boot Applications Demystified
  - Getting Started with Testcontainers for Spring Boot
- Spring Boot testing workshops (in-house/remote/hybrid)
- Consulting offerings, e.g. the Test Maturity Assessment for projects/teams

My Entire Spring Boot Testing Knowledge Combined

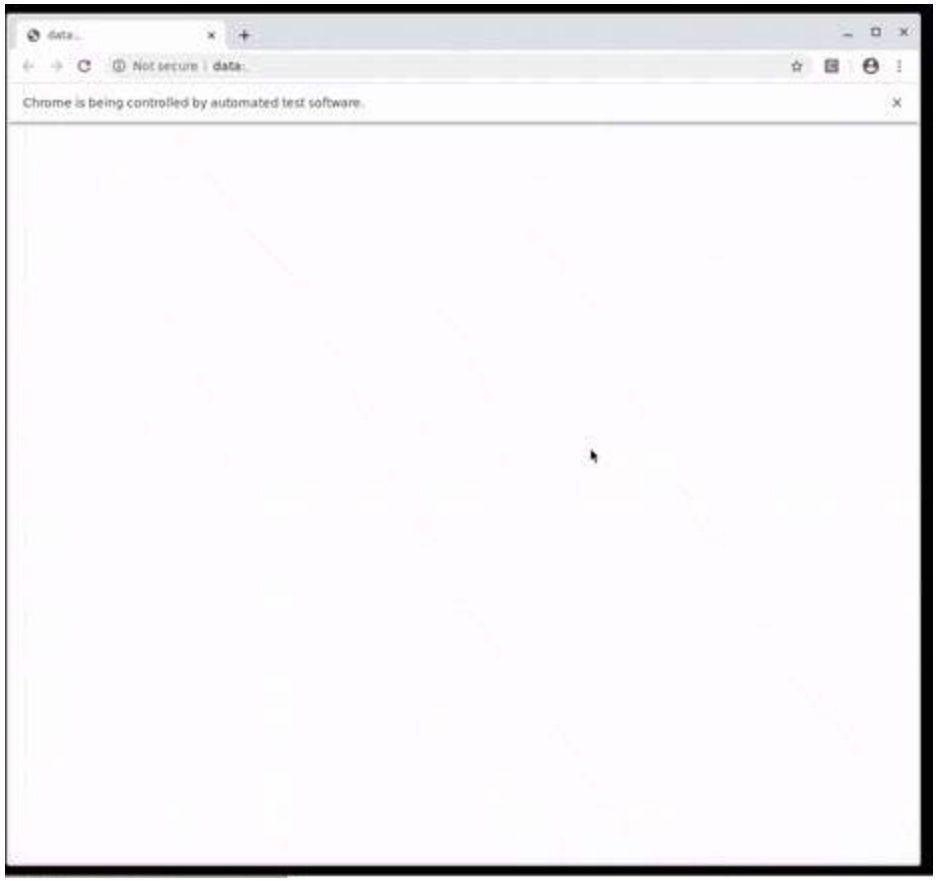
... in one on-demand online course.

Learn how to test and verify a real-world self-contained system with the Testing Spring Boot Applications Masterclass



Covering Unit, Sliced, Integration and E2E Tests

... with 130 course lessons and 12h+ of content



Limited Webinar Offer for the Next 24 Hours

All webinar attendees get a **50% discount** on the course price with this link: https://rieckpil.de/testing-spring-boot-applications-masterclass/?promo=WEBINAR-2025-12-17

Enrolling for the Bundle Edition gives you three additional resources for free:

- TDD with Spring Boot Done Right Online Course
- Hands-On Mocking with Mockito Online Course
- 30 Testing Tools and Libraries Every Java Developer Must Know eBook

The offer expires on the 18th of December 2025 6 PM CET.

Joyful Testing!

Slides & recording will be shared after the webinar via email.

Reach out any time via:

- LinkedIn (Philip Riecks)
- X (@rieckpil)
- Mail (philip@pragmatech.digital)